Week 6 Report

Infrastructure as Code & Configuration Management
DevOps Internship | Davine Technologies

1. Infrastructure as Code (IaC)

Infrastructure as Code means describing servers, networks and other infrastructure in files that can be versioned, reviewed and re-run, instead of clicking through consoles or typing commands by hand. Benefits: repeatable environments, fewer manual errors, version history in Git, easy rebuilds and consistency between dev, test and production.

2. Terraform

Terraform (HashiCorp) is a declarative IaC tool. You describe the desired end state in HCL and Terraform works out what to create, change or destroy. It talks to platforms through **providers** such as AWS, Azure, Docker or local. The core workflow:

Command	Purpose
terraform init	Download providers and modules, initialise the working directory
terraform validate / fmt	Check syntax and format the code
terraform plan	Preview changes without making them
terraform apply	Create or update the resources
terraform destroy	Remove everything Terraform created

```
terraform {
  required_providers { local = { source = "hashicorp/local" } }
}
resource "local_file" "welcome" {
  filename = "${path.module}/output/welcome.txt"
  content  = "Welcome to ${var.project_name}"
}
```

3. Terraform State

The **state file** (terraform.tfstate) records the real resources Terraform manages and maps them to the code. Terraform compares code, state and reality to build a plan. State can contain sensitive values, so it must not be committed to Git; in teams it is stored in a remote backend (S3, Terraform Cloud) with locking. Useful commands: terraform state list, terraform state show.

4. Variables and Outputs

Variables make configurations reusable by moving values out of the code (variables.tf, terraform.tfvars, -var flags). They can have types, defaults and validation rules. **Outputs** print useful values after apply, such as an IP address or URL, and pass values between modules.

```
variable "environment" {
  type    = string
  default = "dev"
  validation {
    condition     = contains(["dev","staging","prod"], var.environment)
    error_message = "environment must be dev, staging or prod."
  }
}
output "welcome_file" { value = local_file.welcome.filename }
```

5. Modules

A **module** is a reusable group of resources with its own inputs and outputs. Calling a module avoids copy-paste and standardises infrastructure. In this week's task a file_generator module creates one config file per application name using for_each.

```
module "app_configs" {
  source      = "./modules/file_generator"
  environment = var.environment
  app_names   = var.app_names
  output_dir  = "${path.module}/output"
}
```

6. Ansible

Ansible is an agentless configuration management tool. It connects over SSH and runs **modules** (apt, user, service, template, file) described in YAML. Its modules are **idempotent**: running a playbook again only changes what is not already in the desired state.

7. Inventory

The **inventory** lists the machines Ansible manages, grouped (for example webservers) with connection details and variables.

```
[webservers]
week6-server ansible_host=127.0.0.1 ansible_port=2222 ansible_user=ansible

[all:vars]
ansible_python_interpreter=/usr/bin/python3
```

8. Playbooks

A **playbook** is a YAML file describing which hosts to configure and the ordered **tasks** to run. **Handlers** run only when notified by a change (for example reload Nginx after a config change). **Templates** (Jinja2) generate files from variables.

```
- hosts: webservers
  become: true
  tasks:
    - name: Install a package
      ansible.builtin.apt: { name: nginx, state: present }
    - name: Create a user
      ansible.builtin.user: { name: appuser, state: present }
    - name: Start the service
```

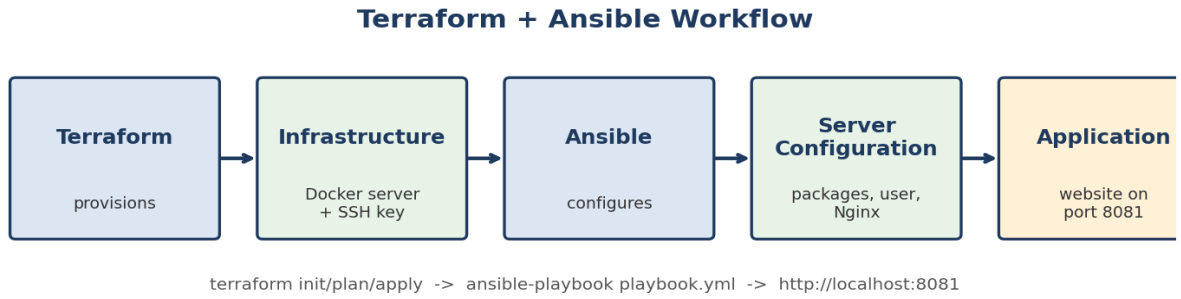
```
ansible.builtin.service: { name: nginx, state: started }
```

9. Terraform vs Ansible

	Terraform	Ansible
Purpose	Provision infrastructure	Configure servers and software
Style	Declarative; tracks state	Task-based; idempotent modules
Language	HCL	YAML
State	terraform.tfstate	No state file
Connection	Provider APIs	SSH, agentless
Example	Create a VM, network or container	Install Nginx, create users, deploy files

Together: Terraform creates the server, then Ansible configures it. Terraform can even generate the Ansible inventory from its own outputs.

10. Hands-on Activity: Terraform + Ansible Workflow



Terraform built and started a Docker-based server with SSH, generated an SSH key and wrote the Ansible inventory. Ansible connected over SSH, installed packages, created an application user, deployed a website and configured and started Nginx. After changing one variable and re-running the playbook, only the website changed; a further run made no changes, confirming idempotency.

11. Best Practices and Conclusion

Practice	Why
Always run terraform plan before apply	Review what will be created, changed or destroyed
Never commit state files or private keys	They can contain secrets
Use variables and modules	Reusable, consistent code
Make Ansible tasks idempotent and use handlers	Safe to re-run; services reload only when needed
Test AI-generated IaC before using it	Prevents unexpected changes to real infrastructure

Week 6 showed how infrastructure can be created and configured automatically. Terraform provisioning and Ansible configuration together give repeatable, scalable and maintainable environments.